
Vibe Coding

From a rough idea to a live app — a builder's field guide to making software by talking to AI.

**Who it's for**

Anyone curious about building apps — no coding background needed

**What's inside**

The idea, the workflow, the journey, best practices, the tools & the building blocks

**You'll need**

A laptop, an internet connection, and an idea to play with

SESSION NOTES

What's inside

A practical walk-through of vibe coding: what it is, how to do it well, the journey from a rough idea to a live app, and the tools and building blocks that make it all work. Written so that *anyone* can follow along, no coding background required.

- 1 What is Vibe Coding?** The idea, where it came from, who it's for
 - 2 How to Vibe Code** The loop, the mindset, prompting like a director
 - 3 The Journey: Idea to App** Ideation, defining features, building
 - 4 Best Practices** Habits that separate shipping from spinning
 - 5 The Tools: Lovable, Bolt & Codex** Deep dives plus a full feature comparison
 - 6 The Building Blocks, Explained Simply** Front end, backend, database, GitHub, skill files, Vercel, Supabase
 - 7 Your First Vibe Coding Session** A starter checklist
-

SECTION 1

What is Vibe Coding?

Vibe coding is a way of building software where you describe what you want in plain English and let an AI write the actual code for you.

Instead of typing out every line yourself, you **steer**. You say what you want, look at what the AI produces, ask for changes in conversation, and repeat until it works. Your job shifts from *writing* code to *directing* the thing that writes it. You stay focused on the idea and the outcome; the machine handles the syntax, the setup, and the plumbing.

The phrase was coined by **Andrej Karpathy**, a co-founder of OpenAI, in early 2025. He described the experience as fully giving in to the vibes and almost forgetting that the code is even there, building an app by simply seeing things, saying things, running things, and adjusting until it mostly works.

The shift in one line

For years, building software meant code, money, and weeks of waiting. Vibe coding compresses that into a conversation: you talk, the AI builds, and a working first version can appear in an afternoon.

Who is it for?

Two very different groups get the most out of it. The first is **experienced developers**, who use it to move fast on prototypes and throwaway ideas without sweating the boilerplate. The second is **complete beginners** — founders, marketers, product managers, students — who have an idea but can't yet code it themselves. One useful way to think about it: vibe coding is for anyone with a *vision they can't execute*, paired with an AI that can.

What vibe coding is *not*

It is not a magic wand, and this is the single most important thing to understand before you start. A working demo is not the same as a finished, safe, reliable product. AI-written code can look completely correct and still break in unexpected situations, leak data, or fall over when lots of people use it at once. Someone still has to check the logic, the security, and whether it will hold up as it grows.

⚠️ The golden rule

Speed is the gift; judgement is the responsibility. Use vibe coding to get to a working version fast, then treat anything you intend to put in front of real users, real money, or real data as something that deserves a careful second look.

Where it's heading

By 2026, even Karpathy had started nudging toward a more precise name: **agentic engineering**. The idea is that you no longer write most of the code directly. Instead you *orchestrate* AI agents that do the work while you provide direction, taste, and oversight. The "vibes" get you moving; the engineering judgement is what makes what you build actually last. Whatever we call it, the direction is the same — describing intent is becoming more important than memorising syntax.

SECTION 2

How to Vibe Code

At its heart, vibe coding is a simple loop you repeat until you're happy.

The core loop

1. **Describe** what you want, in plain language.
2. **Generate** — the AI builds it and shows you a live preview.
3. **Preview** — you look at the result and try it out.
4. **Refine** — you ask for the next change or a fix.
5. **Repeat** — round and round until it does what you imagined.

You are the director

The most useful mental model is this: **you are the director, and the AI is a fast, eager junior developer**. It will build exactly what you ask for, very quickly, without complaint. That cuts both ways. Vague instructions get vague results; clear instructions get sharp ones. The quality of what you get out is mostly a reflection of the clarity you put in.

Prompt like you're briefing a teammate

You don't need special "prompt engineering" tricks. You just need to brief the AI the way you'd brief a capable new colleague who knows nothing about your specific project yet:

- **Give context.** What are you building, and who is it for? "A booking page for a small yoga studio."
- **Set a role.** Telling it to act as a senior developer or a careful designer nudges it toward better output.
- **Be specific about the next step.** Ask for one clear thing, not ten vague ones.
- **Say what "done" looks like.** Describe how you'll know it worked — what you should see, what should happen when you click.

Specific beats vague, every time

✗ *Make me a nice landing page.*

✓ **Specific prompt:** *Build a landing page for a meal-prep service. Top to bottom: a headline and a sign-up button, a three-column "how it works" section, customer reviews, and a footer. Use a clean, warm look with a green accent. Make it work well on mobile.*

Small steps beat giant leaps

The fastest way to get stuck is to ask for everything at once. When you do, errors pile up on top of each other and become a tangle that's hard to undo. Ask for one feature, check that it works, then ask for the next. If something breaks, you'll know exactly which change caused it. Many tools now offer a **plan mode** or **chat mode** that lets the AI think through the whole thing with you *before* it writes any code — use it. A few minutes of planning saves hours of untangling.

SECTION 3

The Journey: From Idea to Live App

Every project moves through the same path. The tools change; the journey doesn't.

The flow runs in four stages: **Ideation → Defining Features → Building → Ship & Iterate**. The first three are where almost all the thinking happens, so we'll spend most of our time there.

1. Ideation

Start with the problem and the person, not the technology. What are you actually trying to solve, and who feels that problem today? Resist the urge to open a tool straightaway. The single most valuable thing you can do here is finish this sentence:

The one-sentence test

"I'm building *[what]* for *[who]*, so they can *[do the one core thing]*."

If you can say that clearly, you're ready. If you can't, you're not stuck on code — you're still figuring out the idea, and no tool will do that part for you.

2. Defining Features

Now turn the idea into a short list of what the app needs to do. The trick is to ruthlessly separate **must-haves** from **nice-to-haves**. Almost everyone wants to build too much at the start. The goal of a first build is not to be complete; it's to be the **smallest version that's still genuinely useful** — what builders call an **MVP** (Minimum Viable Product).

- **Must-have:** a customer can see your classes and book one.
- **Nice-to-have (later):** loyalty points, gift cards, a referral program.

Write your must-haves down as a simple list. That list becomes your build plan, and you'll hand it to the AI one item at a time.

3. Building the App

This is where the loop from Section 2 comes alive. A typical build runs like this:

- **Prompt the first version.** Describe the whole app at a high level and let the tool scaffold it.
- **Preview it.** A live, clickable version appears in seconds. Try it like a real user would.
- **Refine in small steps.** "Make the header sticky." "Change the accent to teal." One change at a time.
- **Connect a backend if you need one.** The moment you need to store information or let people log in, you wire up a database (more on this in Section 6).
- **Test as a real user.** Click everything. Sign up. Break it on purpose. Fix what wobbles.
- **Deploy.** Publish it to a real web address so other people can use it.

4. Ship & Iterate

Getting it live is the start, not the finish. Real feedback from real people beats any amount of guessing in your own head. Watch how people use it, listen to what confuses them, and feed

those learnings back into the loop. The best vibe-coded products are simply the ones that went around this circle the most times.

Notice that only the building stage involves a tool at all. The clarity you bring in stages one and two is what makes the building stage fast – and it's the part the AI can't do for you.

SECTION 4

Best Practices of Vibe Coding

These habits are what separate people who ship working apps from people who go in circles.

1. Plan before you prompt

Use the planning or chat mode in your tool to agree on the approach *before* any code is written. A good sign you're working well: if the AI doesn't ask you any questions about your request, ask it to ask you some.

2. Be specific and concrete

Replace adjectives with details. "Modern" means nothing; "a clean white layout with one green accent colour and lots of spacing" means something the AI can actually build.

3. Change one thing at a time

Small, single-purpose requests are easier for the AI to get right and far easier for you to review. They also make it obvious which change caused a problem when something breaks.

4. Save your work often

Connect your project to **GitHub** (Section 6) so every version is saved and you can roll back to a working state if a change goes wrong. Think of it as an unlimited undo button for your whole project.

5. Test as you build

Don't save all the checking for the end. Try each feature the moment it's added. Bugs are cheap to fix when they're fresh and expensive to untangle once they're buried under later work.

6. Take security seriously

This matters the instant real people or real data are involved. Run the built-in **security scan** if your tool has one, never paste passwords or secret keys directly into prompts or code, and be cautious with anything handling logins or payments.

7. Read what ships — and keep a human in the loop

For anything important, don't blindly accept everything. Skim what changed, sanity-check the behaviour, and for serious projects, get a developer to review before launch. The AI is a brilliant junior, not the final word.

8. Mind your credits and tokens

Most tools charge by usage. Batch your edits, use the free visual-editing modes for small text and colour tweaks, and break big features into smaller steps so a failed attempt doesn't burn through your budget.

⚠ The trap to avoid

The most common and most costly mistake is mistaking a working demo for a finished product. A demo proves the idea. A product survives real users, edge cases, security threats, and growth. Know which one you've built, and be honest about it before you rely on it.

SECTION 5

The Tools: Lovable, Bolt & Codex

Three of the most widely used names in vibe coding — and they sit at three different points on the same spectrum.

Think of a line that runs from "*never touch code*" to "*powerful tool for people who can.*" **Lovable** sits at the friendly end — you can stay in chat and never see code if you don't want to. **Bolt** sits in the middle — chat when you like, jump into the code when you need to, all in your browser. **Codex** sits at the powerful end — an AI agent that works across a real codebase, built mainly for developers. Picking the right one is mostly about where *you* sit on that line.

Lovable — the friendliest way in

Lovable turns plain-English prompts into a complete, working web app and lets you keep refining it through chat. Built by a Swedish company in 2024, it grew remarkably fast and is one of the headline names in the "vibe coding" category. It's often described as a *superhuman full-stack engineer* that can be roughly twenty times faster than coding by hand for straightforward apps.

How it works: you describe your product, and Lovable generates the front end, sets up a backend and database for you, adds things like user logins, and gives you a live preview you can keep shaping by chatting. Its **Visual Edits** mode lets you tweak text, colours, and spacing by

clicking directly on the preview without spending AI credits, while **Dev Mode** lets more confident users edit the code.

Best for: non-technical founders, product managers, and teams who want a polished MVP or prototype fast, with the comfort of staying mostly in conversation.

Watch out for: its usage-based credit pricing can be unpredictable when you're debugging tricky problems, and like all these tools it needs a human review before anything goes to production.

Bolt — the hybrid in your browser

Bolt (bolt.new) is made by StackBlitz and runs an entire development environment *inside your web browser* — a real file system, package manager, and live server, with no installation. That's powered by a clever technology called WebContainers. Like the others, it's driven by leading AI models.

How it works: you describe your app, and Bolt builds the whole project, installs everything it needs, and shows a live preview *next to* the editable code. This side-by-side approach is its signature: beginners can stay in the chat, while developers can dive into the files whenever they want. Its **Plan Mode** outlines the full project before writing code, which saves time and budget.

Best for: founders building prototypes and demos, hackathons, and people who want the option to see and edit real code without leaving the browser.

Watch out for: it's cloud-only (no offline mode), it can struggle with complex custom logic, and heavy debugging can run up token costs.

Codex — the developer's AI agent

Codex is OpenAI's agentic coding system, and it's a different kind of animal from the other two. Rather than a "type a prompt, get a website" builder, it's an AI *agent* that works across an entire real codebase: it reads your project, makes changes, runs tests, and hands back results. It lives in several places that share one underlying model — a command-line tool, an extension inside code editors, inside ChatGPT, a connection to GitHub, and desktop apps for Mac and Windows.

How it works: you assign it a task and it works in an isolated, safe sandbox (a private copy of your code), often running several tasks in parallel. It can plan multi-step work, fix failing tests on its own, and even includes a security agent that finds and fixes vulnerabilities.

Best for: developers and technical teams who already have a codebase and want to delegate real engineering work — this is "agentic engineering" in its fullest form.

Watch out for: it assumes comfort with developer concepts and workflows, so it's the steepest of the three for an absolute beginner. It's also tied to OpenAI's own models.

Full feature comparison

Here's how the three line up across the features that matter most. Where a tool isn't built for something, the table says so plainly rather than pretending otherwise.

Feature	Lovable	Bolt (bolt.new)	OpenAI Codex
Best described as	Prompt-to-app builder	Browser-based hybrid builder	Agentic coding system
Made by	Lovable (founded 2024)	StackBlitz	OpenAI
Underlying AI	Anthropic Claude	Claude & other top models	OpenAI's GPT-5 series (Codex)
Who it's for	Non-technical builders	Founders & developers alike	Developers & technical teams
Where it runs	In the browser	In the browser (WebContainers)	Terminal, editor, ChatGPT, cloud, desktop
How you interact	Chat & visual editing	Chat <i>and</i> live code editor	Assign tasks to an AI agent
Builds the front end	✓ Yes	✓ Yes	✓ Via code in your repo
Backend & database	✓ Built in / via Supabase	✓ Bolt Cloud / via Supabase	✓ Works in your existing codebase
User logins (auth)	✓ Yes	✓ Yes	✓ If your project uses it
One-click deploy	✓ With custom domain	✓ Via Netlify / Vercel	Via your own pipeline, not one-click
Hosting included	✓ Yes (+ Lovable Cloud)	✓ Bolt Cloud / Netlify	No — you choose your host
Saves to GitHub	✓ Full two-way sync	✓ Import & export	✓ Deep GitHub integration
Mobile apps	Web-first	✓ Via Expo (React Native)	✓ If your codebase targets mobile
Import from Figma	Limited	✓ Yes (design to code)	Not a design tool
Payments (Stripe)	✓ Via integration	✓ Built-in integration	✓ Via code
Visual / no-credit edits	✓ Visual Edits mode	✓ Preview editor	No — code only
Plan-before-build mode	✓ Chat Mode Agent	✓ Plan Mode	✓ Plans multi-step tasks
Built-in security scan	✓ Yes (with Supabase)	Via integrations	✓ Dedicated security agent
Runs tasks in parallel	One project at a time	One project at a time	✓ Multiple agents at once

Feature	Lovable	Bolt (bolt.new)	OpenAI Codex
Team collaboration	✓ Multiplayer workspaces	✓ Team workspaces	✓ Enterprise & team features
You own the code	✓ Yes, no lock-in	✓ Yes, no lock-in	✓ It's your repo
Pricing model	Monthly + usage credits	Monthly + usage tokens	Included with ChatGPT plans
Easiest for beginners	✓✓ Most beginner-friendly	✓ Middle ground	Steepest learning curve

Features and pricing for all three tools change frequently — always check each product's own site for the latest. Snapshot reflects publicly available information in 2026.

Which one should I pick?

If you've **never written code** and want a polished app fast → start with **Lovable**.

If you want speed **but also the option to peek at and edit the code** → reach for **Bolt**.

If you're a **developer with a real codebase** to work on → that's **Codex**.

SECTION 6

The Building Blocks, Explained Simply

You'll hear these seven words constantly. Here's what each one really means — no jargon.

The first three — front end, backend, and database — are easiest to understand together, so picture a **restaurant**.

1. Front End

The front end is **everything the user sees and touches**: the buttons, text, images, colours, and layout. In our restaurant, this is the *dining room* — the part guests actually experience. When people say an app "looks good," they're talking about the front end. It's what shows up in your web browser or on your phone screen.

2. Backend

The backend is the **engine working behind the scenes** that you never see. It handles the rules, the logic, and the heavy lifting: checking your password is correct, calculating a total, sending a confirmation email. In the restaurant, this is the *kitchen* — guests don't go in, but nothing arrives at the table without it.

3. Database

The database is **where all the information is stored and remembered** — user accounts, orders, messages, every saved thing. Without it, an app would forget everything the moment you closed it. In the restaurant, this is the *pantry and store-room*: the organised shelves the kitchen pulls from and restocks.

*Put together: you tap a button in the **dining room** (front end), the **kitchen** (backend) does the work, and it fetches what it needs from the **pantry** (database). That's essentially every app you've ever used.*

4. GitHub

GitHub is a **safe home for your code in the cloud**, with a complete history of every change. The simplest way to picture it is *"Google Docs for code, with a built-in time machine."* Every version is saved, you can rewind to any earlier point if something breaks, and a developer can pick up your project from it later. Crucially, because your code lives there, you're never locked in to a single tool — you genuinely own your work.

5. Skill files (for better, more consistent results)

A "skill" file — you'll see names like **SKILL.md**, **AGENTS.md**, or a project's rules file — is a simple text document that **teaches the AI your standards and preferences**, so it builds things your way without being reminded every time. Think of it as a *briefing document or style guide you hand to a new team member on day one*.

Inside, you might write things like "always use our brand colours," "keep the design clean and minimal," "use this framework, not that one," or "follow these rules for writing safe code." Once it's set up, the AI reads it automatically and stays consistent across everything it makes. The **.md** simply means it's a Markdown file — plain text with very light formatting, easy for anyone to write. Good skill files are one of the biggest upgrades you can make to both the *design* and the *quality* of what you build.

6. Vercel (for deployment)

Deployment is the act of **putting your app live on the internet** so anyone can visit it. **Vercel** is a popular service that does this for you: you connect your project and it publishes your site to a real web address, fast, and re-publishes automatically whenever you make changes. Think of it as the *"publish" button for your website* — it takes what you built and makes it reachable by the whole world.

7. Supabase (for the backend)

Supabase is a **ready-made backend in a box**. Instead of building the engine and the store-room from scratch, you plug in Supabase and instantly get a database, user logins, file storage, and

the connections an app needs to talk to them. It's why these vibe-coding tools can give your app real, working features — saving data, signing people in — so quickly. Picture it as a *kitchen-and-pantry that arrives fully fitted*, so you can focus on the dining room.

Quick reference

Term	In plain English	The analogy
Front End	What users see and click	The dining room
Backend	The behind-the-scenes engine and logic	The kitchen
Database	Where information is stored and remembered	The pantry
GitHub	Cloud home for code, with full history	Google Docs + a time machine
Skill files	Instructions that teach the AI your standards	A briefing / style guide
Vercel	Publishes your app live to the internet	The "publish" button
Supabase	A ready-made backend (database, logins, storage)	A fully-fitted kitchen

SECTION 7

Your First Vibe Coding Session

A short checklist to take you from blank screen to a live, working first version.

🎯 Before you open a tool

- ✓ Finish the one-sentence test: *I'm building [what] for [who] so they can [the one core thing].*
- ✓ Write a short list of **must-have** features, and a separate "later" list for everything else.
- ✓ Pick your tool: **Lovable** to stay in chat, or **Bolt** if you'd like to see the code too.

While you build

- ✓ Use **plan mode** first — agree on the approach before any code is written.
- ✓ Prompt for the **whole app once**, then refine **one change at a time**.
- ✓ Be specific: describe layout, colours, and what each button should do.
- ✓ Connect **Supabase** when you need to store data or let people log in.
- ✓ Connect **GitHub** so every version is saved and reversible.
- ✓ Test each feature as you add it — click everything.

Before you share it

- ✓ Run the **security scan** if your tool offers one.
- ✓ **Deploy** with Vercel (or your tool's built-in publish) to get a real web address.
- ✓ For anything serious, have someone **review it** before real users, money, or data are involved.

One last thing

The only way to get good at vibe coding is to build something. Start with something small and slightly silly — the stakes are low and the lessons are real. Describe it, watch it appear, change it, break it, fix it. Go around the loop a few times and the workflow stops feeling like magic and starts feeling like a tool you reach for without thinking. That's the whole game.

"Vibe coding is having a vision you can't execute — paired with an AI that can." The vision is yours to bring. Everything in these notes is just about getting better at handing it over clearly.